

EOD Task Completion Report – Git & GitHub Assignment

Date: 19-Aug-2026

Task: Git & GitHub Basics

Status: Completed

Jira Ticket: Created for the assignment

1. Objective

The objective of this assignment was to understand the basics of Git and GitHub, create and secure a GitHub account, configure Git, and practice commonly used Git commands for version control and repository management.

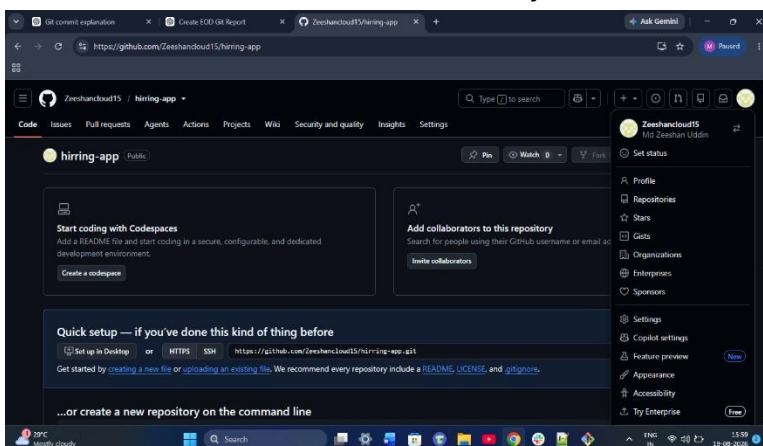
2. Video Watched and Notes Prepared

I watched the assigned video and prepared notes covering:

- Git and GitHub basics
 - Git repository initialization
 - Working directory and staging area
 - Git configuration
 - Adding and committing files
 - Viewing commit history
 - Connecting a local repository to GitHub
 - Pushing code to a remote repository
 - Pulling and fetching changes
 - Cloning repositories
 - Creating and using SSH keys
 - Reverting commits
 - Git remote repository management
-

3. GitHub Account Creation

Created a GitHub account successfully.



Steps performed:

1. Created the GitHub account.
 2. Logged in successfully.
 3. Verified access to the GitHub dashboard.
 4. Prepared the account for repository and Git operations.
-

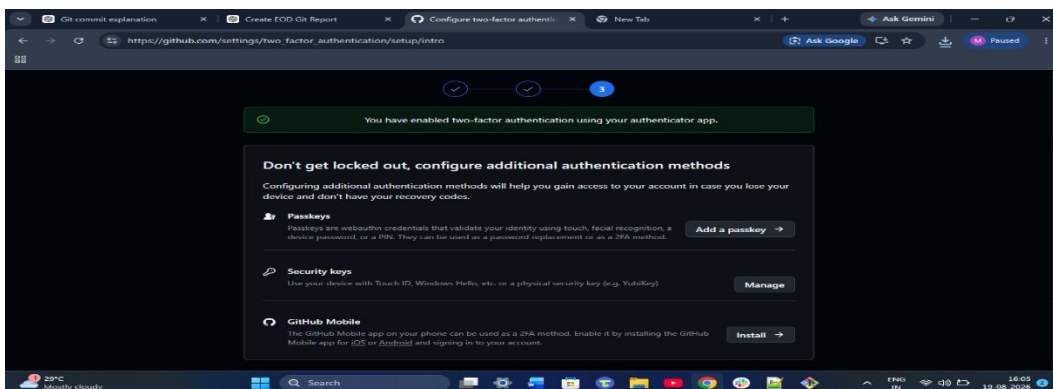
4. MFA Configuration

Enabled Multi-Factor Authentication (MFA) for the GitHub account using a mobile authenticator application.

Steps performed:

1. Opened GitHub account settings.
2. Navigated to password/security settings.
3. Selected two-factor authentication.
4. Configured the authenticator mobile application.
5. Scanned the QR code.
6. Verified the authentication code.
7. Successfully enabled MFA.

Evidence: MFA enabled screenshot



5. Git Commands Executed

1 Check Git Version

Command:

`git --version`

Purpose: Checks whether Git is installed and displays the installed Git version.

2 Check Repository Status

Command:

`git status`

Purpose: Displays the current state of the Git working directory and staging area.

3 Initialize Git Repository

Command:

`git init`

Purpose: Initializes a new Git repository in the current directory.

4 List Hidden Files

Command:

`ls -a`

Purpose: Displays all files, including hidden files such as `.git`.

5 Create a File

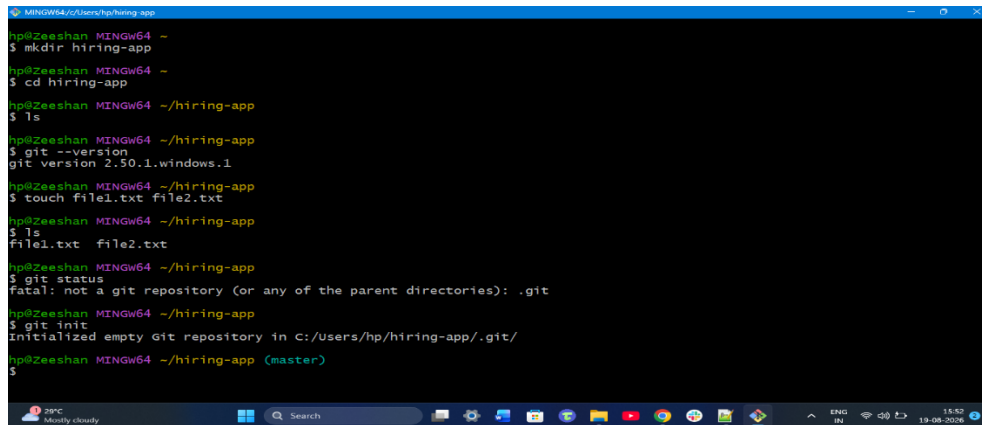
Command:

touch filename

Purpose: Creates a new empty file.

Example:

touch test.txt



```
hp@Zeeshan MINGW64 ~
$ mkdir hiring-app
hp@Zeeshan MINGW64 ~
$ cd hiring-app
hp@Zeeshan MINGW64 ~/hiring-app
$ ls
hp@Zeeshan MINGW64 ~/hiring-app
$ git --version
git version 2.50.1.windows.1
hp@Zeeshan MINGW64 ~/hiring-app
$ touch file1.txt file2.txt
hp@Zeeshan MINGW64 ~/hiring-app
$ ls
file1.txt  file2.txt
hp@Zeeshan MINGW64 ~/hiring-app
$ git status
fatal: not a git repository (or any of the parent directories): .git
hp@Zeeshan MINGW64 ~/hiring-app
$ git init
initialized empty git repository in c:/Users/hp/hiring-app/.git/
hp@Zeeshan MINGW64 ~/hiring-app (master)
$
```

6 Remove Files/Directories

Command:

rm -rf filename

Purpose: Removes files or directories recursively and forcefully.

Note: This command should be used carefully because deleted files may not be recoverable through normal means.

7 Check Git Configuration

Command:

git config --list

Purpose: Displays the configured Git settings.

8 Configure Git Username

Command:

git config --global user.name "zeeshan"

Purpose: Sets the Git username globally.

9 Configure Git Email

Command:

git config --global user.email "zeeshan@123"

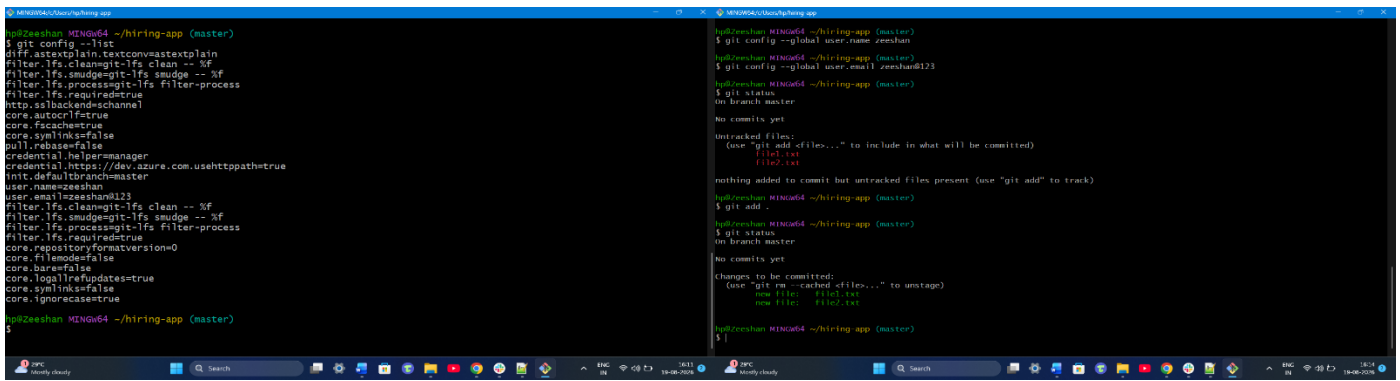
Purpose: Sets the email address associated with Git commits.

10 Add Files to Staging Area

Command:

git add .

Purpose: Adds modified and new files to the Git staging area for the next commit.



```
hp@Zeeshan MINGW64 ~/hiring-app (master)
$ git config --list
diff.autostash true
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.postBuffer=65536
core.autocrlf=true
core.fsmonitor=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.helper=manager
init.defaultBranch=master
user.name=zeeshan
user.email=zeeshan123
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
core.repositoryformatversion=0
core.filemode=false
core.bare=false
core.logallfileupdates=true
core.symlinks=false
core.ignorecase=true

hp@Zeeshan MINGW64 ~/hiring-app (master)
$

hp@Zeeshan MINGW64 ~/hiring-app (master)
$ git config --global user.name zeeshan
hp@Zeeshan MINGW64 ~/hiring-app (master)
$ git config --global user.email zeeshan123
hp@Zeeshan MINGW64 ~/hiring-app (master)
$ git status
On branch master
no commits yet

Untracked files:
  (use "git add <files>..." to include in what will be committed)
    file1.txt
    file2.txt

nothing added to commit but untracked files present (use "git add" to track)
hp@Zeeshan MINGW64 ~/hiring-app (master)
$ git add .
hp@Zeeshan MINGW64 ~/hiring-app (master)
$ git status
On branch master
no commits yet

Changes to be committed:
  (use "git rm --cached <files>..." to unstage)
    new file:   file1.txt
    new file:   file2.txt

hp@Zeeshan MINGW64 ~/hiring-app (master)
$
```

11 Create a Commit

Command:

`git commit -m "Initial commit"`

Purpose: Saves the staged changes into the local Git repository with a commit message.

12 View Commit History

Command:

`git log`

Purpose: Displays the commit history of the repository.

6. Connect Local Repository to GitHub

Add Remote Repository

Command:

`git remote add origin <repository-url>`

Purpose: Connects the local Git repository to the remote GitHub repository.

Example:

`git remote add origin https://github.com/username/repository.git`

Push Local Repository to GitHub

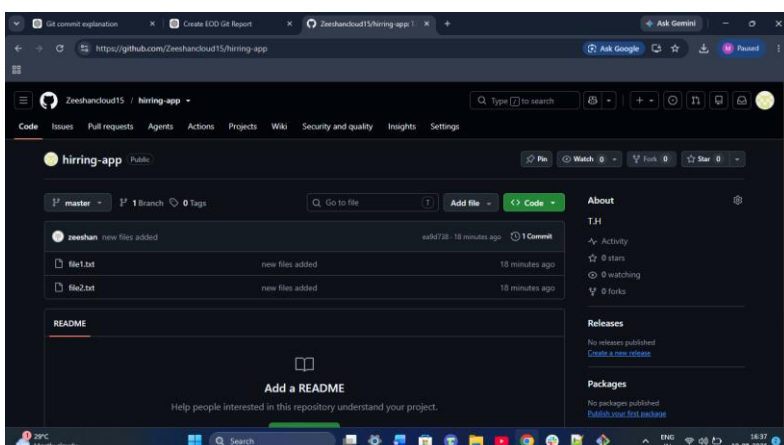
Command:

`git push -u origin main`

Purpose: Pushes the local main branch to the GitHub remote repository and establishes the upstream relationship.

Evidence: GitHub repository showing pushed files/commits

Screenshot:



7. SSH Key Generation

Command:

ssh-keygen

Purpose: Generates an SSH key pair that can be used for secure authentication with GitHub.

Steps performed:

1. Generated an SSH key pair.
2. Located the public key.
3. Added the public key to GitHub.
4. Used SSH authentication for Git operations.

Evidence: Terminal/GitHub SSH key configuration screenshot

Screenshot:

```
MINGW64/Users/hiring_app
```

```
m0Zeehan MINGW64 ~/hiring-app(master)  
$ git remote add origin https://github.com/Zeeshancloud15/hiring-app.git  
  
m0Zeehan MINGW64 ~/hiring-app(master)  
$ ssh-keygen  
Generating public/private ed25519 key pair.  
Enter file in which to save the key (/C:/Users/hp/.ssh/id_ed25519):  
.C:/Users/hp/.ssh/id_ed25519 already exists:  
Override (y/n)? y  
Enter passphrase for "/C:/Users/hp/.ssh/id_ed25519" (empty for no passphrase):  
Enter same passphrase again:  
Your identification has been saved in /C:/Users/hp/.ssh/id_ed25519  
Your public key has been saved in /C:/Users/hp/.ssh/id_ed25519.pub  
The key fingerprint is:  
SHA256:gYqSGhSWHwVcyjFRmkJk54LekXzqcU9NgcO+sonGMM hp@Zeeshan  
The key's randomart image is:  
----[ED25519 256]-----  
          |  
        .++      +  
         ++..    +  
        ..+++   +  
       + O ooi+  +..  
     +- S=+o -+-  
    + O oo+= +=+  
  + O . . O + .oo|  
+ O .-.- O+.e.|  
  +O -. O+.O.  
-----[SHA256]-----  
  
m0Zeehan MINGW64 ~/hiring-app(master)  
$ cat ./C:/Users/hp/.ssh/ided25519.pub  
sh-ed25519 AAAAC3NzaC1lZDI1NTU5AAAIxakMzn7YkfIXcyc36krFvr+zIzAFjvIJGUHFhgBosmo hp@Zeeshan  
  
m0Zeehan MINGW64 ~/hiring-app(master)  
$ cat ./C:/Users/hp/.ssh/id_id.ed25519.pub  
sh-ed25519 AAAAC3NzaC1lZDI1NTU5AAAIxakMzn7YkfIXcyc36krFvr+zIzAFjvIJGUHFhgBosmo hp@Zeeshan  
  
m0Zeehan MINGW64 ~/hiring-app(master)
```

8. Git Pull

Command:

```
git pull
```

Purpose: Downloads changes from the remote repository and integrates them into the current local branch.

9. Git Clone

Command:

```
git clone <repository-url>
```

Purpose: Creates a local copy of an existing remote Git repository.

Example:

<https://github.com/Zeeshancloud15/hirring-app.git>

10. Git Fetch

Command:

```
git fetch
```

Purpose: Downloads the latest changes from the remote repository without automatically merging them into the current branch.

11. Check Remote Repository

Command:

```
git remote -v
```

Purpose: Displays the URLs configured for the remote repository.

Example output:

origin <https://github.com/Zeeshancloud15/hirring-app.git> (fetch)

origin <https://github.com/Zeeshancloud15/hirring-app.git> (push)

12. Git Revert

Command:

`git revert <commit-id>`

Purpose: Creates a new commit that reverses the changes introduced by an earlier commit.

Key point: git revert does not remove the existing commit from history. It creates a new commit that reverses its changes.

13. Commands/Topics Covered

Command	Purpose
<code>git --version</code>	Check Git version
<code>git status</code>	Check repository status
<code>git init</code>	Initialize repository
<code>ls -a</code>	Show hidden files
<code>touch</code>	Create file
<code>rm -rf</code>	Remove files/directories
<code>git config --list</code>	View Git configuration
<code>git config</code>	Configure Git user details
<code>git add</code>	Add changes to staging
<code>git commit -m</code>	Create commit
<code>git log</code>	View commit history
<code>git remote add origin</code>	Add remote repository
<code>git push</code>	Upload changes to remote repository
<code>git push -u origin main</code>	Push and set upstream
<code>ssh-keygen</code>	Generate SSH keys
<code>git pull</code>	Download and integrate changes
<code>git clone</code>	Copy remote repository locally
<code>git fetch</code>	Download remote changes
<code>git remote -v</code>	View remote URLs
<code>git revert</code>	Reverse a previous commit

16. Final Status

Task Status: COMPLETED

The assigned Git and GitHub activities were completed successfully. Git was configured, a GitHub account was created and secured with MFA, and the required Git commands were practiced and verified.

EOD Submission: Completed and ready for review.